

Design Definition File (DDF)

Flight Software for AOCS Subsystem

CubeSat Team PoliTo

December 30, 2025

Contents

1	Introduction	4
2	Software Architectural Design (Static)	5
2.1	Software Layers	5
2.2	Component Decomposition	5
3	Software Architectural Design (Dynamic)	6
3.1	Task Overview	6
3.1.1	CHECK/SAFETY Task	6
3.2	Scheduling Logic	7
3.3	Synchronization Strategy	7
4	Software Detailed Design	8
4.1	Mode Manager State Machine	8
4.2	Control Algorithms	8
4.3	OS Design	8
4.3.1	FreeRTOS Configuration	8
5	Interfaces	9
5.1	Internal Interfaces	9
5.2	External Interfaces	9

List of Figures

3.1	CHECK Task Flowchart	7
3.2	short	7

List of Tables

2.1	Software Components by Layer	5
3.1	Task overview with priorities and roles	6

Chapter 1

Introduction

The document aims to describe features, architecture and implementation for the AOCS Flight Software developed for the ELECTRA 3U Mission.

The document adopts the ECSS-E-ST-40C Rev.1 Standard.

Chapter 2

Software Architectural Design (Static)

2.1 Software Layers

The firmware is divided into two distinct layers, isolating hardware-independent from hardware-specific logic:

- **Blue Layer (Drivers / HAL)**: handles direct hardware interaction and abstraction.
- **Red Layer (Application / Components)**: contains the AOCS-specific logic.

2.2 Component Decomposition

The following components are identified in each layer:

Component / File	Description
Blue Layer (Drivers / HAL)	
UARTdriver.h/.c	Custom library enhancing standard USART capabilities for packet handling.
bufferUtils.h/.c	Manages circular buffers for serial communication.
frameUtils.h/.c	Implements frame search algorithms within serial buffers.
queue.h, adc.h, spi.h	Standard low-level peripheral drivers generated by STM32CubeMX.
Red Layer (Application / Components)	
MTi.h/.c	Manages UART communication with the Xsens IMU.
actuator_drivers.h/.c	Manages PWM signals for magnetorquers and reads current via internal ADC.
sensors.h/.c	Handles SPI communication with the external ADC/MUX for NTC temperature sensors.
pid_conversions.h/.c	Implements the PD control law (error → torque → dipole → current → duty cycle).

Table 2.1: Software Components by Layer

Chapter 3

Software Architectural Design (Dynamic)

3.1 Task Overview

The application is split into 4 main FreeRTOS tasks with specific priorities:

Task Name	Priority	Role
CHECK/SAFETY	High	Monitors health data. Checks for actuator over-currents and IC overheating. It can trigger interrupt routines to enter safe states (e.g., reducing PWM duty cycle).
CONTROL	Normal	The core logic. Executes the attitude control algorithm (PD controller). It handles 3 OpModes: IDLE (0), CONTROL (1), and TEST/POLARITY (2).
OBC-COMM	Normal	Handles communication with the On-Board Computer (OBC). Receives target data and sends telemetry.
IMU	Normal	Samples angular speed and magnetic field from the IMU and sends data to Control and OBC tasks.

Table 3.1: Task overview with priorities and roles

3.1.1 CHECK/SAFETY Task

The CHECK/SAFETY Task performs the following operations.

Actuator Overcurrent

The task monitors all the actuators current (3 magnetorquers), ensuring that enough current is driven to every driver.

IC Overheating

The task monitors the temperature of critical ICs, as the CPU and actuators. If the CPU exceeds a critical temperature the system MUST be put in a temporary IDLE/SAFE state. If actuators overheat, the amount of current is reduced by modifying the Duty Cycle of the PWM signal for the specific actuator.

The device enters a safe state whenever a temperature overcome or an overcurrent happen. This event MUST be avoided to not endanger the system during the mission.

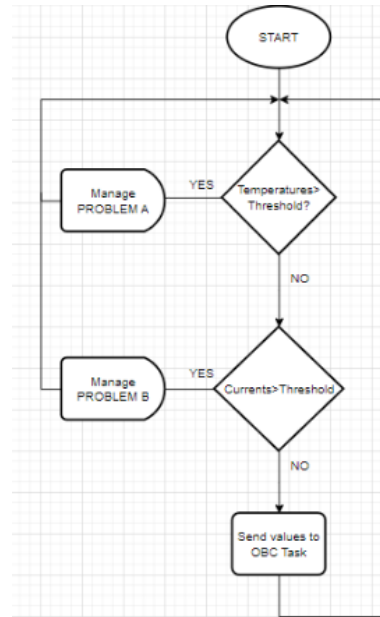


Figure 3.1: CHECK Task Flowchart

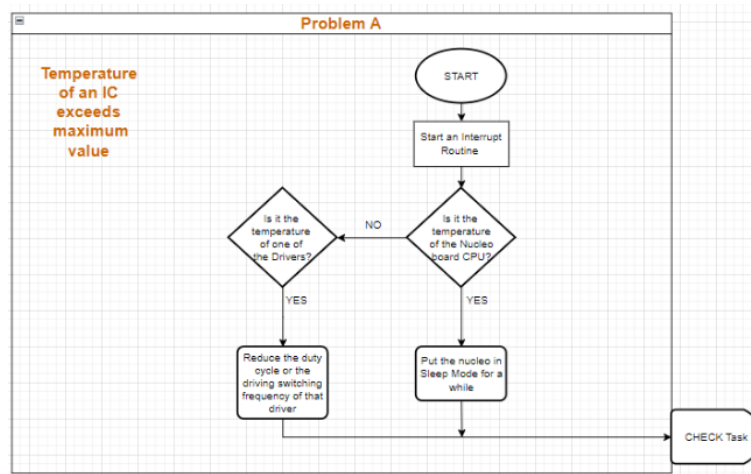


Figure 3.2: Problem A Flowchart

3.2 Scheduling Logic

3.3 Synchronization Strategy

Chapter 4

Software Detailed Design

4.1 Mode Manager State Machine

The section aims to describe the possible transitions in the system.

4.2 Control Algorithms

4.3 OS Design

4.3.1 FreeRTOS Configuration

In `FreeRTOSConfig.h`, key configuration parameters are:

- `configUSE_PREEMPTION`: 1 (Enables preemptive scheduling)
- `configTICK_RATE_HZ`: 1000 (Scheduler frequency)
- `configMINIMAL_STACK_SIZE`: 128
- `configTOTAL_HEAP_SIZE`: 3000

The Heap 4 memory management model is adopted.

Chapter 5

Interfaces

5.1 Internal Interfaces

Data is exchanged via FreeRTOS Queues:

- i) IMUQueue1, IMUQueue2:* Send sensor data from the IMU Task to the Control and OBC tasks.
- ii) setAttitudeADCSQueue:* Sends target attitude data from the OBC Task to the Control Task.
- iii) ADCSHouseKeepingQueue:* Sends current and temperature telemetry from the Check Task to the OBC Task
- iv) SetOpModeADCSQueue:* Used to change the operational mode (Idle, Control, Test)

5.2 External Interfaces

UART and SPI